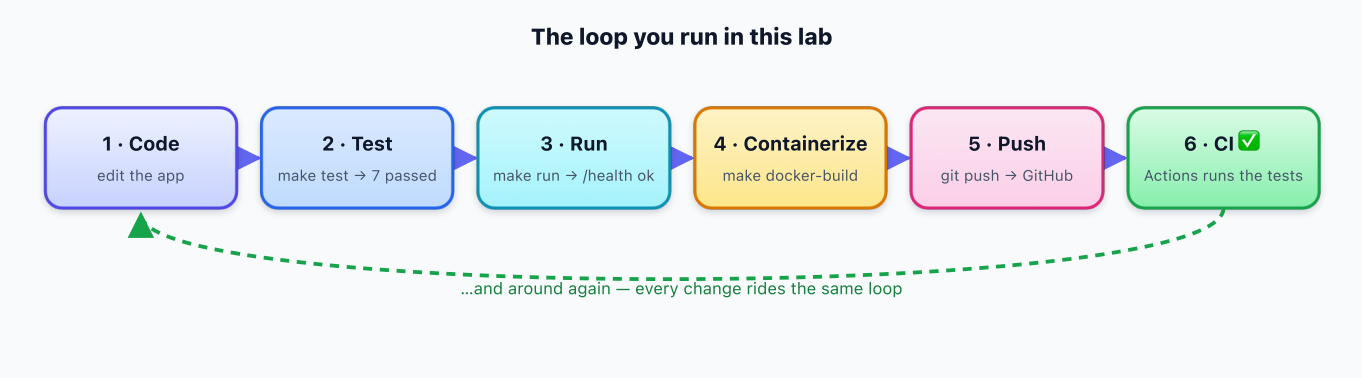
Week 0 — Lab: Set Up Your Toolchain

> 🧪 **Hands-on setup for Week 0.** For the plain-language concept primer and key terms, see **week-00-notes.md**. Do this lab *before* Week 1.

🧪 Lab: From Zero to a Running, Tested, Containerized App

Time budget: ~2 hours (less if you use Codespaces).

Goal: Prove your toolchain works by taking one small real project — the starter service — all the way through the DevOps loop: **run it → test it → containerize it → push it → watch CI go green → point an AI agent at it.**



What you submit: a short checklist with a few screenshots (see Step 8).

> **Two ways to do this lab.** Pick one:
>
> - **Path A — Codespaces (easiest, nothing to install):** everything runs in your browser. Jump to Step 1B.
> - **Path B — Local install:** install the tools on your own machine. Start at Step 1A.
>
> If you've never set up a dev environment before, **use Path A.**

Step 0: Create a GitHub account (both paths)

1. Sign up (free) at https://github.com/join.
2. You'll use this account to host your repo and run CI.

Step 1A: (Path B) Install locally

Install three things, then verify each one. **Verifying is the important part** — "installed" is not "working."

Git

- macOS: `git` is usually present; if not, install Xcode Command Line Tools: `xcode-select --install`
- Windows: download from https://git-scm.com/download/win

- Linux: `sudo apt-get install -y git`

```
git --version          # expect: git version 2.x
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
```

Python 3.12+

- Download from https://www.python.org/downloads/ (or use your OS package manager).

```
python3 --version      # expect: Python 3.12.x or newer
```

Docker Desktop (optional but recommended — needed for the container step)

- Download from https://www.docker.com/products/docker-desktop/
- Start Docker Desktop, then:

```
docker --version       # expect: Docker version 2x.x
docker run hello-world # prints a success message if Docker is working
```

➡️ Now go to Step 2.

Step 1B: (Path A) GitHub Codespaces

No local installs. Python and Docker come pre-configured via the starter's Dev Container.

1. Open the course repository on GitHub.
2. Click the green **< > Code** button → **Codespaces** tab → **Create codespace**.
3. Wait ~1 minute while it builds. You get a full VS Code editor in your browser with a terminal.
4. In the terminal: `cd project/starter`

➡️ Now skip to Step 3 (the dependencies were installed for you by `postCreateCommand`).

Step 2: Get the starter project

```
# Clone the course repository (replace with your course's repo URL)
git clone https://github.com/<your-course-org>/CSE636.git
cd CSE636/project/starter
```

Take 2 minutes to read `project/starter/README.md` so you know what you're about to run.

Step 3: Run the starter service

These three commands are the heart of the lab.

```
make setup     # creates a virtualenv and installs Flask + pytest
make test      # runs the test suite — you should see "7 passed"
make run       # starts the web app at http://localhost:8000
```

With the app running, open a **second terminal** (or browser) and check the endpoints:

```
curl http://localhost:8000/health
# -> {"status":"ok"}

curl "http://localhost:8000/risk?
files_changed=3&lines_changed=120&tests_passing=true"
# ->
{"files_changed":3,"lines_changed":120,"risk_score":0.27,"tests_passing":tr
ue}
```

Stop the app with **Ctrl+C** when you're done.

> 🎉 If you saw `7 passed` and `{"status":"ok"}`, you've just run the full inner DevOps loop: code → test → run.

▶ ✅ Did it work? Check your output

Step 4: Containerize it (needs Docker)

```
make docker-build     # builds an image named cse636-starter
make docker-run       # runs the app in a container at
http://localhost:8000
```

Check `http://localhost:8000/health` again — same result, but now it's running *inside a container*. That's the "works the same everywhere" promise of Docker. Stop it with **Ctrl+C**.

> On Codespaces this works thanks to docker-in-docker. If `docker` isn't available, skip this step and note it in your submission.

Step 5: Make it *your* repo and push it

You'll push the starter as your **own** new repository so that GitHub runs CI for you.

1. On GitHub, click **New repository**. Name it `cse636-starter`. Leave it empty (no README). Create it.
2. Back in your terminal, from inside `project/starter`:

```
# Start a fresh repo from just the starter folder
rm -rf .git                       # detach from the course repo's history
git init
git add .
git commit -m "Initial commit: CSE636 starter service"
git branch -M main
git remote add origin https://github.com/<your-username>/cse636-starter.git
git push -u origin main
```

> **Why `rm -rf .git`?** GitHub only runs workflow files that sit at the **repo root**. By pushing just the starter folder as its own repo, the included `.github/workflows/ci.yml` lands at the root, where Actions will find it.

Step 6: Watch CI go green

1. Open your new repo on GitHub.
2. Click the **Actions** tab.
3. You should see a workflow run named **CI** triggered by your push.
4. Click it and watch it install dependencies and run the tests. It should finish with a **green checkmark** ✅.

> This is Continuous Integration: you pushed code, and a fresh machine in the cloud automatically tested it. **Take a screenshot of the green run** for your submission.

Optional — see CI catch a failure. On a branch, break a test on purpose (change an expected number in `tests/test_main.py`), commit, push, and open a pull request. Watch CI turn **red** ❌ on the PR. Then fix it and watch it go green. This is exactly the feedback loop Week 3 builds agents around.

▶ ✅ Did it work? No CI run appears in the Actions tab

Step 7: Point an AI coding agent at the repo

Install **one** agent and give it a single task. (Week 1 goes much deeper; here we just confirm it runs.)

Option A — Claude Code (terminal-based):

```
# Requires Node.js (https://nodejs.org). Then:
npm install -g @anthropic-ai/claude-code
export ANTHROPIC_API_KEY="your-key-from-https://console.anthropic.com"
cd cse636-starter     # your repo
claude
```

Option B — GitHub Copilot (in VS Code): install VS Code, sign in to the GitHub Copilot extension, and enable **Agent mode** in Copilot settings.

Give the agent this task and watch what it does — do not blindly accept changes:

```
Read this repository and give me a plain-English summary of what the
application does, what each file is for, and how I would run the tests.
```

Notice: the agent *reads files* (a tool), *reasons*, and *reports back*. That perceive → reason → act loop is the thing this whole course is about.

> ⚠️ **Never commit your API key.** Keep it in your shell or a `.env` file — the starter's `.gitignore` already ignores `.env`.

Step 8: What to submit

A short checklist (half a page is fine) confirming you're ready for Week 1:

1. Which path you used (Codespaces or local) and your OS.
2. The output of `make test` (paste the `7 passed` line).
3. A screenshot of your **green CI run** on GitHub (Step 6).
4. The link to your `cse636-starter` repo.
5. One or two sentences: which AI agent you ran, the task you gave it, and **one thing it did that surprised you**.
6. Confirm the self-check boxes in week-00-notes.md are all ticked.

⛑️ Troubleshooting

Symptom	Likely cause	Fix
`make: command not found` (Windows)	`make` isn't installed	Use Git Bash + `choco install make`, or run the commands inside the Makefile by hand, or use Codespaces
`python3: command not found`	Python not installed or not on PATH	Reinstall from python.org and check "Add to PATH"; or use `python` instead of `python3`
`pytest` can't find `app`	Ran pytest outside the starter folder	Run `make test` from inside `project/starter`; the included `pyproject.toml` sets the path
`docker: Cannot connect to the Docker daemon`	Docker Desktop isn't running	Start Docker Desktop and retry; or skip the container step
Push rejected / "remote already exists"	Repo wired to the wrong remote	`git remote remove origin` then re-add your repo URL
No CI run appears in Actions tab	Workflow not at repo root	Make sure you pushed the *starter folder* as its own repo (Step 5), so `.github/` is at the root
Agent asks for an API key	Not authenticated	Set `ANTHROPIC_API_KEY` (Claude Code) or sign in to Copilot

> Stuck for more than 20 minutes on setup? Post in the course channel with the exact command and error message, and bring it to the first class. Don't let setup block you from the Week 1 concepts.